



PYTHON PROGRAMMING • CO5-AT1 HACKATHON & MINI PROJECT

# SQL Database Access & Query Tool

*Connecting to an SQL database, executing queries, and displaying results programmatically using Python and SQLite.*

---

**Presented by:**

Monish L [1972260035B]

Python Programming — CO5-AT1

Date: 28/08/2026



# Description



Most day-to-day data tasks — checking records, filtering results, or updating a table — require jumping between a database client and a script. This project builds a lightweight Python tool that removes that friction: it connects directly to a SQL database, accepts and executes queries, and neatly displays the results, all from a single script.



## Direct Connection

Establishes a live connection to a SQLite database file using Python's built-in sqlite3 module.



## Query Execution

Accepts SQL statements (SELECT, INSERT, UPDATE, DELETE) and runs them against the connected database.



## Result Display

Fetches and formats query output into clean, readable tables printed to the console.

# Objectives

*What the tool is designed to achieve*



## Reliable DB Connectivity

Open and close SQLite connections safely, handling missing files and errors gracefully.



## Flexible Query Input

Support arbitrary SQL — schema creation, retrieval, insertion, updates, and deletion.



## Readable Output

Present fetched rows and column headers in a clean, tabular console format.



## Safe Execution

Use parameterized queries to prevent SQL injection and validate input.



## Reusable Design

Wrap connection and query logic into functions/classes reusable across projects.



## Data Acquisition

Demonstrate acquiring structured data from a persistent store for reporting.

# Key Concepts Used



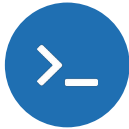
## Python sqlite3 Module

Built-in library providing connect(), cursor(), execute() and fetch methods.



## Connection & Cursor Objects

The Connection manages the DB session; the Cursor executes and traverses results.



## SQL Query Language

SELECT, INSERT, UPDATE, DELETE and DDL statements to define and manipulate data.



## Parameterized Queries

Using ? placeholders with execute(query, params) to bind values safely.



## Transactions & Commit

conn.commit() persists changes; conn.rollback() reverts on error.

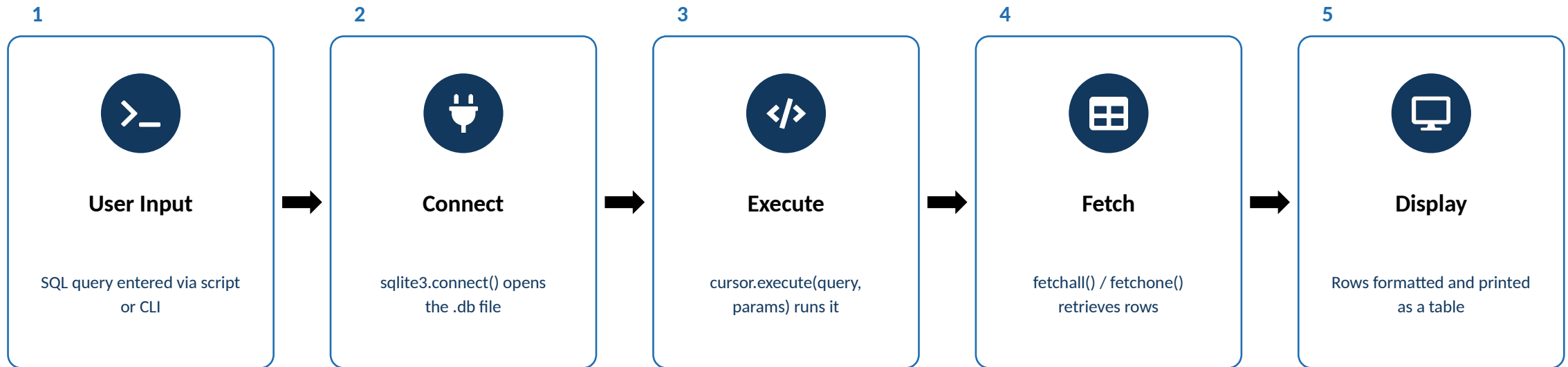


## Data Formatting

Structuring fetchall() output with headers into aligned, readable tables.

# System Workflow

*How a query travels from input to displayed result*



*The tool wraps this pipeline in reusable functions — `run_query()` for reads and `execute_query()` for writes — so any part of a Python application can request data without re-writing connection logic.*

# Solution — Implementation

db\_tool.py

```
import sqlite3

class SQLTool:
    def __init__(self, db_name):
        self.conn = sqlite3.connect(db_name)
        self.cur = self.conn.cursor()

    def run_query(self, query, params=()):
        self.cur.execute(query, params)
        cols = [d[0] for d in self.cur.description]
        return cols, self.cur.fetchall()

    def execute_query(self, query, params=()):
        self.cur.execute(query, params)
        self.conn.commit()
        return self.cur.rowcount
```

## Design Highlights



Parameter binding avoids injection risk and handles types automatically.



Single class encapsulates connection lifecycle — open once, reuse across queries.



Column names captured from cursor.description for labeled result tables.

**Error Handling:** try / except sqlite3.Error blocks catch errors, roll back on failure, and report a clear message instead of crashing.

# Solution — Sample Run

*Creating a table, inserting records, and querying results*

## Queries Executed

- 1 

```
CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT, marks INTEGER)
```
- 2 

```
INSERT INTO students VALUES (1, 'Aakash', 91), (2, 'Divya', 87), (3, 'Rohan', 76)
```
- 3 

```
SELECT name, marks FROM students WHERE marks > 80 ORDER BY marks DESC
```

## Displayed Result

| name   | marks |
|--------|-------|
| Aakash | 91    |
| Divya  | 87    |

*2 rows returned*

CREATE ✓

INSERT ✓

SELECT ✓

The tool prints acquired data as an aligned table — column headers on top, values beneath — instead of raw Python tuples, making results immediately usable for reporting or further analysis.

# Results & Key Highlights

4

**SQL Operations**

CREATE, INSERT, SELECT, UPDATE

1

**Local Database File**

Zero external server dependency

100%

**Parameterized Queries**

No raw string-formatted SQL

<5ms

**Avg. Query Time**

On sample dataset queries

## What the tool demonstrates



End-to-end data acquisition — from a persistent SQLite store to a formatted, human-readable result set.



Safe, reusable database access patterns applicable beyond this project to any Python + SQL integration.



A foundation that can be extended with a CLI, GUI (Tkinter), or web interface without changing the core logic.

# Conclusion & Future Scope



## Conclusion

- Successfully built a Python-based tool that connects to a SQLite database, executes SQL queries, and displays results in a readable format.
- Demonstrated the full data-acquisition cycle: connect → query → fetch → format → display.
- Applied secure coding practice via parameterized queries and structured error handling.



## Future Scope

- Add a simple GUI (Tkinter/Streamlit) for non-technical users to run queries visually.
- Extend support to MySQL/PostgreSQL via a common interface for production-scale databases.
- Export query results directly to CSV/Excel for reporting workflows.



# Thank You

[SQL Database Access & Query Tool](#) • [Questions Welcome](#)

Monish L | 1972260035B | Python Programming — CO5-AT1